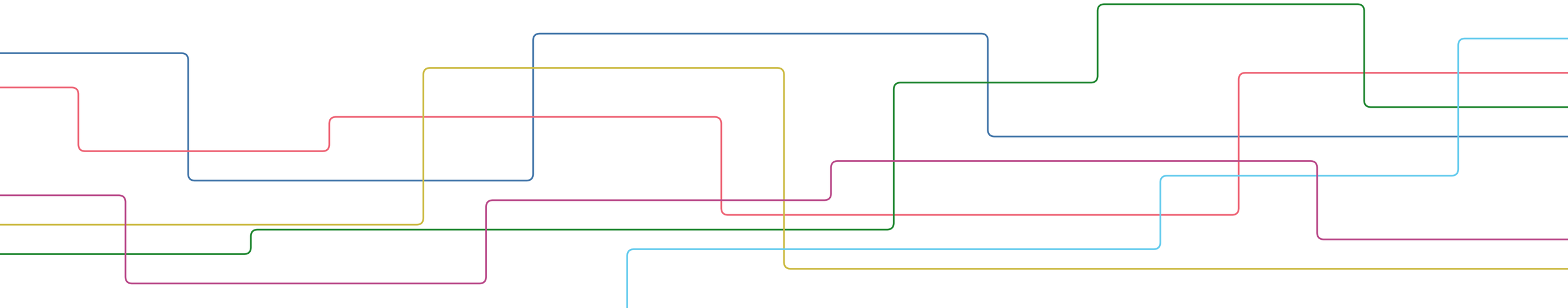
Programming for Data Science

– Feature Learning with Neural Networks

Martin Trapp

Department of Computing and Learning Systems
School of Electrical Engineering and Computer Science
KTH Royal Institute of Technology
mtrapp@kth.se



Outline

Recap Lectures 5 — 6

- Linear regression and ridge regression
- Bayesian linear regression and Gaussian processes

Random Fourier Features

- Monte Carlo integration
- From the RBF kernel to random Fourier features

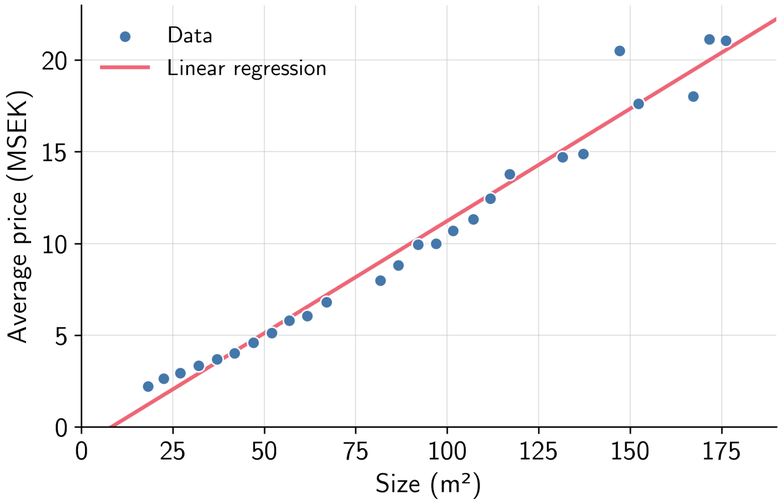
Neural Networks (basics)

- ReLU neural networks
- ReLU networks as decision trees and feature learners
- Manifold hypothesis

Learning neural networks

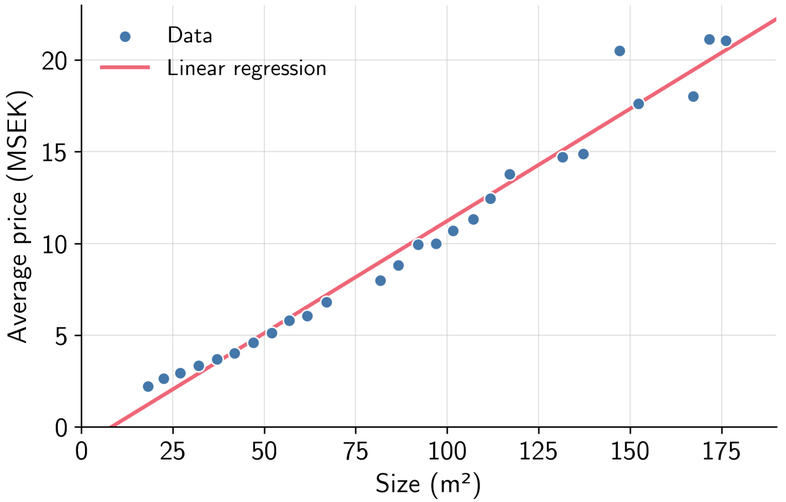
- Gradient descent & chain rule

Recap: Lectures 5 — 6



- **Problem setting:** Learn a regression function
- **Linear regression:** $f(\mathbf{x}) = \mathbf{x}^T\mathbf{w}$
- **Observation:** Predictions in linear regression are based on the similarity (dot product) between the new input and the training data.
- We can change how similarities are calculated by using a feature map $\phi(\,\cdot\,)$.
- **Kernel trick**: For every valid (positive semi-definite) kernel k, there exists a feature map ϕ with $k(\mathbf{x}_1, \mathbf{x}_2) = \phi(\mathbf{x}_1)^T\phi(\mathbf{x}_2)$, so we can work with k directly and never compute ϕ.

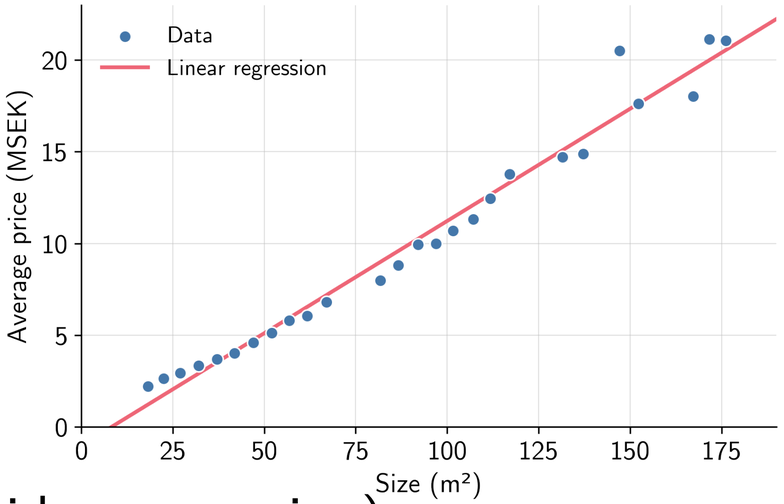
Recap: Lectures 5 — 6



- **Kernel regression:** Instead of $f(\mathbf{x}) = \mathbf{x}^T\mathbf{w}$ we use $f(\mathbf{x}) = \phi(\mathbf{x})^T\mathbf{w}$ but without computing $\phi(\mathbf{x})$ explicitly.
- **Predictions only depend on k:** We compute $f(\mathbf{x}_\star) = \sum_{n=1}^{N} k(\mathbf{x}_\star, \mathbf{x}_n)\left[\mathbf{K}^{-1}\mathbf{y}\right]_n$
- **Problem:** The matrix $\mathbf{K}$ has to be invertible, and predictions are sensitive to outliers.
- **Solution:** We regularize $\mathbf{w}$, which regularizes the matrix $\mathbf{K}$ (ridge regression).

$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} k(\mathbf{x}_\star, \mathbf{x}_n)\left[(\mathbf{K}+\lambda\mathbf{I})^{-1}\,\mathbf{y}\right]_n$$

Recap: Lectures 5 — 6



- **Problem:** The matrix **K** has to be invertible, and predictions are sensitive to outliers.
- **Solution:** We regularize **w**, which regularizes the matrix **K** (ridge regression).

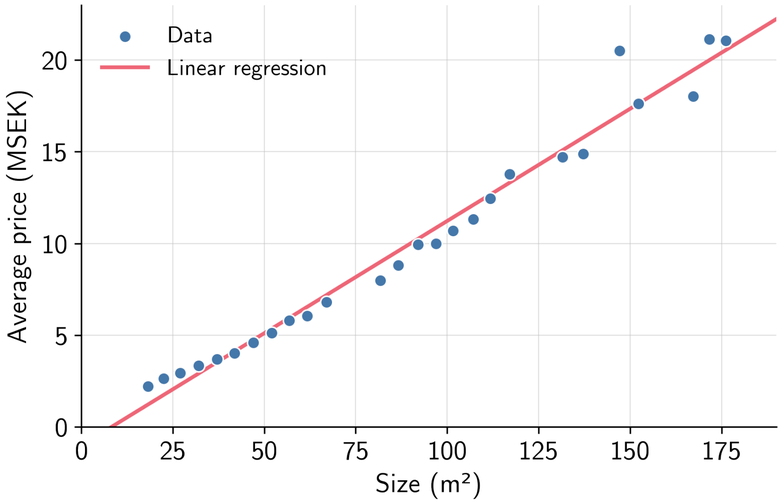
$$f(\mathbf{x}_\star) = \sum_{n=1}^{N} k(\mathbf{x}_\star, \mathbf{x}_n) \left[(\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{y} \right]_n$$

- **Found by minimizing:** $\mathscr{L}(\mathbf{w}) = \sum_{n=1}^{N} \left(y_n - \phi(\mathbf{x}_n)^T \mathbf{w} \right)^2 + \lambda \|\mathbf{w}\|^2$

Regularization

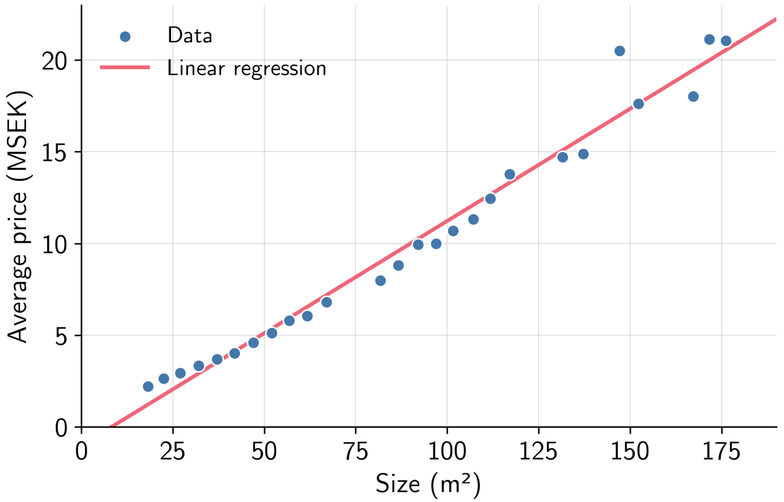
Squared loss

Recap: Lectures 5 — 6



- **Bayesian linear regression:** Use a prior distribution to regularise the values of **w** to be close to zero.
- **Ridge regression:** L2 regularization on **w**, i.e., $\sum_{n=1}^{N} \left(y_n - \phi(\mathbf{x}_n)^T \mathbf{w}\right)^2 + \lambda \|\mathbf{w}\|^2$
- **Gaussian prior:** In Bayesian linear regression, we use $p(\mathbf{w}) = \mathcal{N}(0, \sigma_w^2 \mathbf{I})$.
- Notice that $-\log \mathcal{N}(\mathbf{w} \mid \mathbf{0}, \sigma_w^2 \mathbf{I}) = \frac{1}{2\sigma_w^2} \|\mathbf{w}\|^2 + \text{const}$

Recap: Lectures 5 — 6



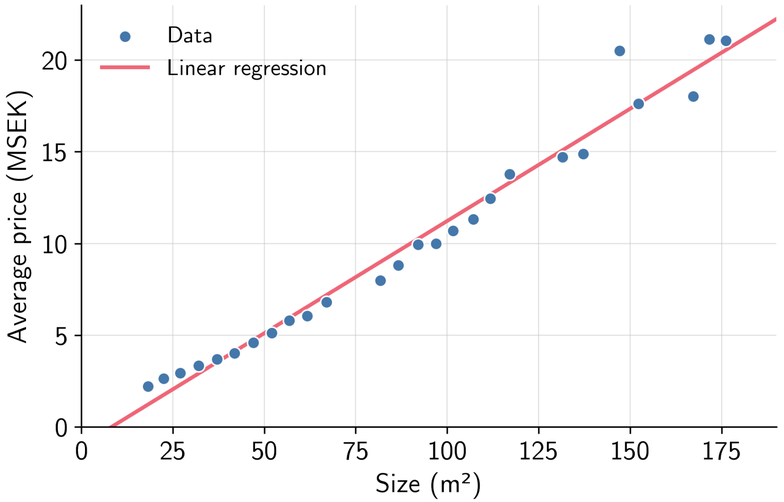
- **Key idea:** No single model anymore, but instead we have a (weighting) distribution $p(\mathbf{w} \mid \mathbf{X}, \mathbf{y})$ over all models parameterized by $\mathbf{w}$.

- **Bayesian model average:** To make predictions, we average over all models $\mathbf{w}_j$, each weighted by its probability $p(\mathbf{w}_j \mid \mathbf{X}, \mathbf{y})$, i.e.,

$$p(y_\star \mid \mathbf{x}_\star, \mathbf{X}, \mathbf{y}) = \int p(y_\star \mid \mathbf{x}_\star, \mathbf{w})\, p(\mathbf{w} \mid \mathbf{X}, \mathbf{y})\, d\mathbf{w}$$

- **What is $p(\mathbf{w}_j \mid \mathbf{X}, \mathbf{y})$?** We found that $p(\mathbf{w}_j \mid \mathbf{X}, \mathbf{y})$ is again a Gaussian distribution.

Recap: Lectures 5 — 6

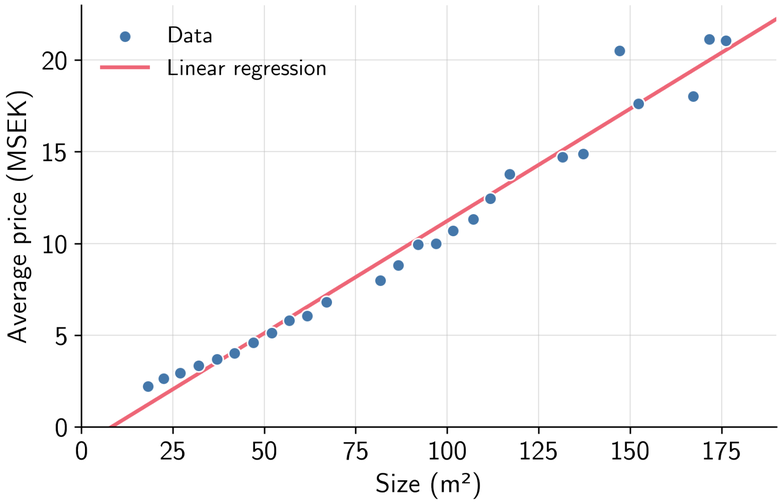


- **Predictions:** Again, use dot products, and we can apply the kernel trick as before.
- As a result, we get the following predictive distribution:

$$\mathcal{N}\left(y_\star \mid \mathbf{k}_\star{}^T \left(\mathbf{K}+\sigma^2\mathbf{I}\right)^{-1}\mathbf{y},\, k(\mathbf{x}_\star,\mathbf{x}_\star)-\mathbf{k}_\star{}^T \left(\mathbf{K}+\sigma^2\mathbf{I}\right)^{-1}\mathbf{k}_\star+\sigma^2\right)$$

- **Problem:** The posterior over **w** lives in feature space, which can be infinite-dimensional, but predictions only need $k(\cdot\,,\cdot)$.
- **Key idea:** We can place a prior distribution on the functions $p(f)$ instead of **w**.

Recap: Lectures 5 — 6

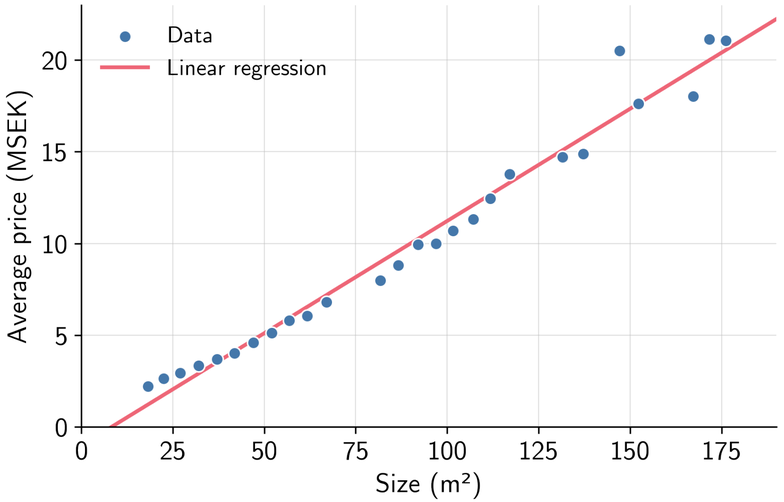


- **Gaussian process:** A stochastic process where any finite marginal distribution is multivariate Gaussian distributed.

- A **Gaussian process** is the natural choice for a prior on f if we assume that predictions (function values) are Gaussian-distributed.

- **Key idea:** Parameterize Gaussians with a mean function $\mathbb{E}[f(\mathbf{x})] = m(\mathbf{x})$ and a covariance (kernel) function $\mathrm{cov}(f(\mathbf{x}), f(\mathbf{x}')) = k(\mathbf{x}, \mathbf{x}')$.

Recap: Lectures 5 — 6



- **Key idea:**

  - **Gaussian process:** —> prior on f
  - **Bayesian linear regression:** —> prior on $\mathbf{w}$

- The predictive distributions of Bayesian linear regression and Gaussian processes are identical if $k(\mathbf{x}, \mathbf{x}') = \sigma_w^2 \phi(\mathbf{x})^T \phi(\mathbf{x}')$.

- **Why use a Gaussian process:** We can directly specify the prior on the functions (e.g., smooth or periodic functions) by choosing a kernel, without ever constructing $\phi(\mathbf{x})$, even if it is infinite-dimensional.

Open questions

- **Natural follow-up questions are:**

1. Can we approximate $k(x, x')$ with a finite-dim. $\phi(x)$?
   *$\phi(x)$ for the RBF kernel spans an infinite-dimensional space.

2. Can we learn $\phi(x)$ from data?

3. If we learn $\phi(x)$ from data, can we get $k(x, x')$?

Open questions

- **Natural follow-up questions are:**

1. Can we approximate $k(x, x')$ with a finite-dim. $\phi(x)$?
   *$\phi(x)$ for the RBF kernel spans an infinite-dimensional space.

   Random features

2. Can we learn $\phi(x)$ from data?

   Neural networks

3. If we learn $\phi(x)$ from data, can we get $k(x, x')$?

   Lecture 8

Random features

- **Recall:** We had that $k(\mathbf{x}_1, \mathbf{x}_2) = \phi(\mathbf{x}_1)^T \phi(\mathbf{x}_2)$.

- If we do predictions, we have to compute:
$$\mathcal{N}\left(y_\star \mid \mathbf{k}_\star^T \left(\mathbf{K} + \sigma^2 \mathbf{I}\right)^{-1} \mathbf{y},\, k(\mathbf{x}_\star, \mathbf{x}_\star) - \mathbf{k}_\star^T \left(\mathbf{K} + \sigma^2 \mathbf{I}\right)^{-1} \mathbf{k}_\star + \sigma^2\right)$$

- **Problem:** Computing $\left(\mathbf{K} + \sigma^2 \mathbf{I}\right)^{-1}$ costs $\mathcal{O}(N^3)$, where N is the number of training examples.

- If $N = 60{,}000$ then we need $N^3 \approx 2 \times 10^{14}$ operations to invert $\mathbf{K} + \sigma^2 \mathbf{I}$, which is roughly 20 minutes on a laptop, and storing $\mathbf{K}$ in memory requires approx. 29 GB.

Random features - Bochner's theorem

- **Recall the RBF kernel:** given as $k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right)$

- **Observation:** The RBF kernel $k(\mathbf{x}, \mathbf{x}')$ is shift-invariant, and we can also write it as $k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|^2}{2\ell^2}\right) = \sigma_f^2 \exp\left(-\frac{\|\boldsymbol{\Delta}\|^2}{2\ell^2}\right) = k(\underbrace{\mathbf{x} - \mathbf{x}'}_{=\boldsymbol{\Delta}})$.

- **Bochner's theorem:** Any shift-invariant kernel can be written in the Fourier domain as an expectation over a spectral density $p(\mathbf{w})$:

$$k(\boldsymbol{\Delta}) = \sigma_f^2 \, \mathbb{E}_{\mathbf{w} \sim p(\mathbf{w})} \left[\cos\left(\mathbf{w}^T \boldsymbol{\Delta}\right)\right] = \sigma_f^2 \int_{\mathbf{w}} p(\mathbf{w}) \cos\left(\mathbf{w}^T \boldsymbol{\Delta}\right) \, d\mathbf{w}$$

Random features - Bochner's theorem

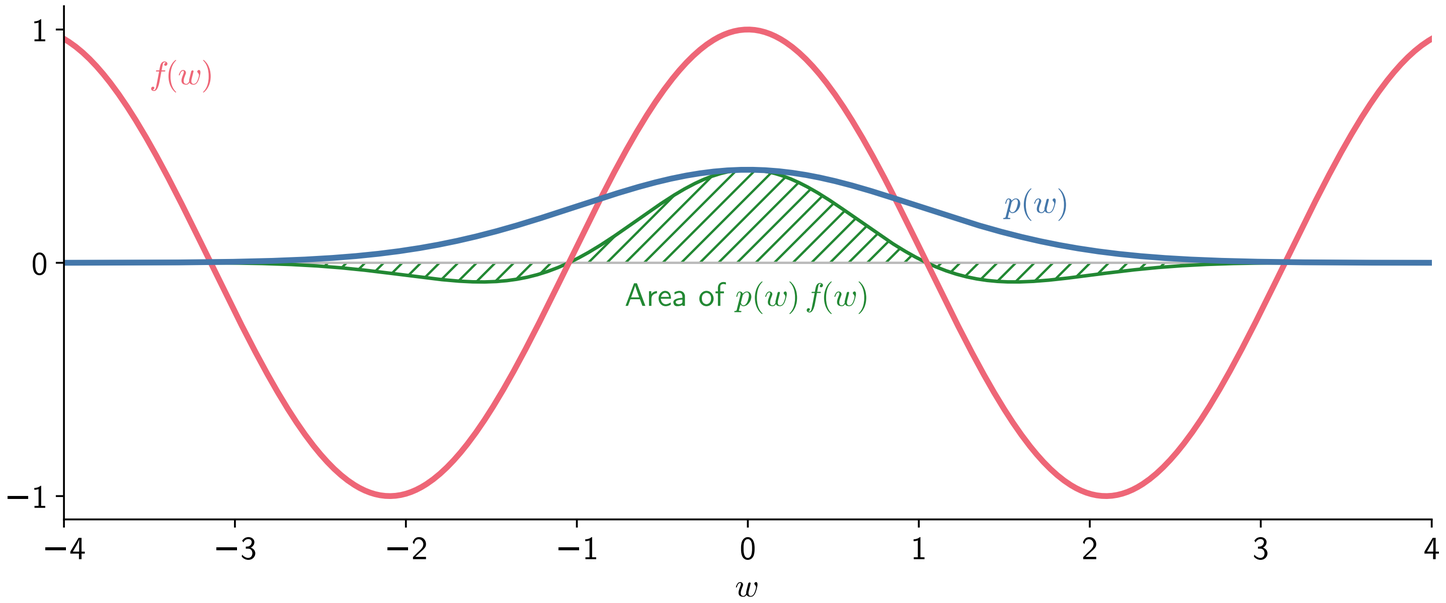
- **Bochner's theorem:** Any shift-invariant kernel can be written in the Fourier domain as an expectation over a spectral density $p(\mathbf{w})$:

$$k(\boldsymbol{\Delta}) = \sigma_f^2 \, \mathbb{E}_{\mathbf{w} \sim p(\mathbf{w})} \left[\cos\left(\mathbf{w}^T \boldsymbol{\Delta} \right) \right] = \sigma_f^2 \int_{\mathbf{w}} p(\mathbf{w}) \cos\left(\mathbf{w}^T \boldsymbol{\Delta} \right) \, d\mathbf{w}$$

- **Examples:**
  - **RBF kernel:** The spectral density is $p(\mathbf{w}) = \mathcal{N}\left(\mathbf{w} \mid \mathbf{0}, \ell^{-2} \mathbf{I} \right)$
  - **Laplace kernel:** The spectral density is $p(\mathbf{w}) = \prod_{d=1}^{D} \frac{\ell}{\pi \left(1 + \ell^2 w_d^2 \right)}$

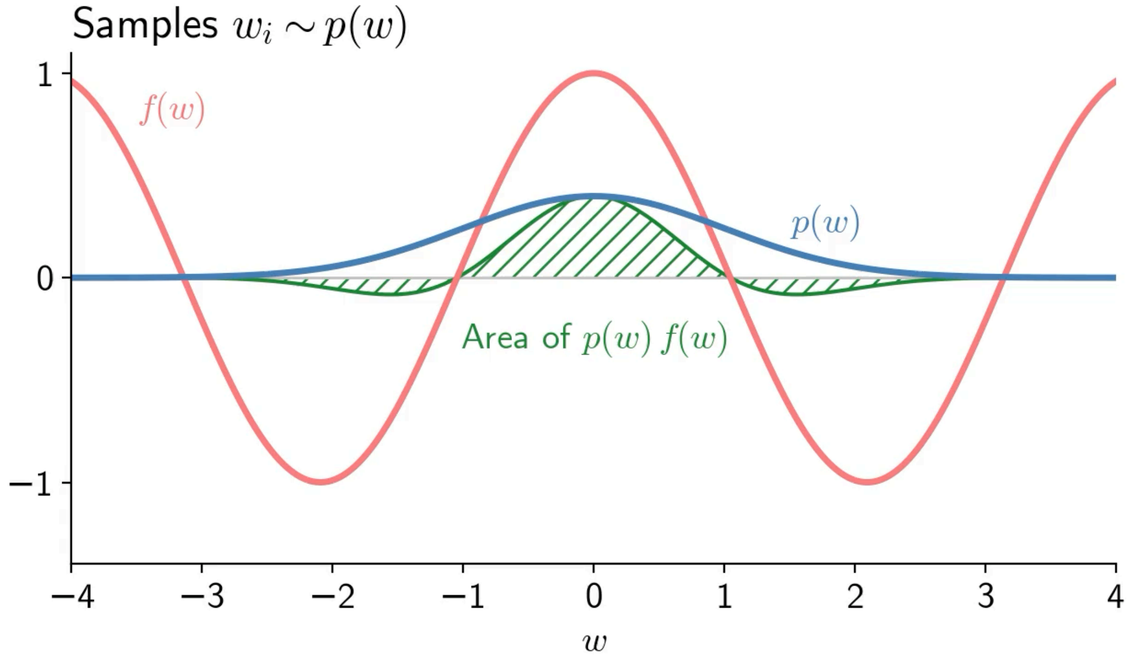
Monte Carlo integration

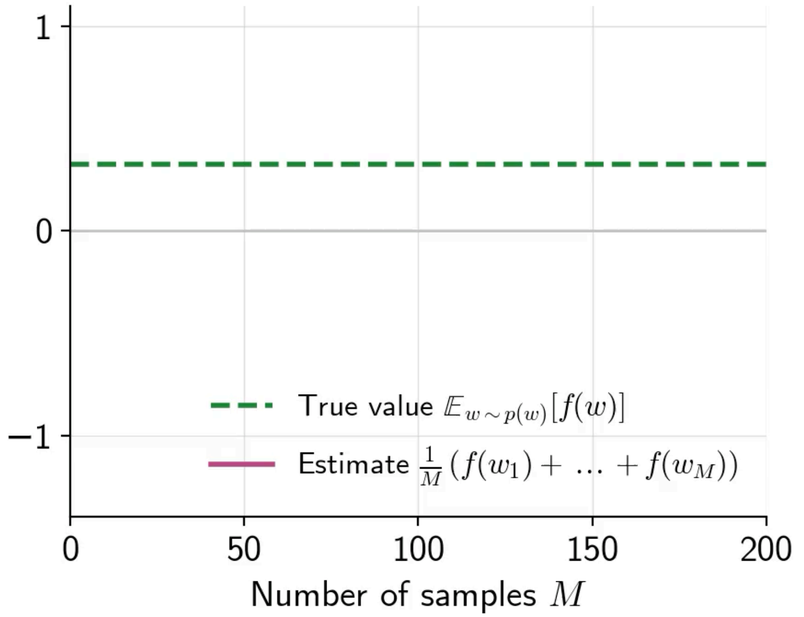
- **Given:** An expectation we aim to compute $\mathbb{E}_{w\sim p(w)}[f(w)]$
- **Goal:** We want to approximate the expectation.
- **Solution:** Use Monte Carlo integration $\mathbb{E}_{w\sim p(w)}[f(w)] \approx \frac{1}{M}\sum_{i=1}^{M} f(w_i),\ w_i \sim p(w)$



Monte Carlo integration

- **Given:** An expectation we aim to compute $\mathbb{E}_{w\sim p(w)}[f(w)]$
- **Goal:** We want to approximate the expectation.
- **Solution:** Use Monte Carlo integration $\mathbb{E}_{w\sim p(w)}[f(w)] \approx \frac{1}{M}\sum_{i=1}^{M} f(w_i),\ w_i \sim p(w)$





Random Fourier features

- **RBF kernel:** $k(\boldsymbol{\Delta}) = \sigma_f^2 \, \mathbb{E}_{\mathbf{w} \sim p(\mathbf{w})} \left[\cos\left(\mathbf{w}^T \boldsymbol{\Delta} \right) \right]$ with $p(\mathbf{w}) = \mathcal{N}\left(\mathbf{w} \mid \mathbf{0}, \ell^{-2} \mathbf{I} \right)$
- **How can we approximate the expectation?**
- **Key idea:** Apply Monte Carlo integration!

Random Fourier features

- **Random Fourier features:**
  - Draw $\mathbf{w}_1, \ldots, \mathbf{w}_M \sim \mathcal{N}(\mathbf{0}, \ell^{-2}\mathbf{I})$
  - Then $k(\boldsymbol{\Delta}) \approx \frac{\sigma_f^2}{M} \sum_{i=1}^{M} \cos(\mathbf{w}_i^T \boldsymbol{\Delta})$.
  - Use that $\cos(\mathbf{w}_i^T(\mathbf{x} - \mathbf{x}')) = \cos(\mathbf{w}_i^T \mathbf{x})\cos(\mathbf{w}_i^T \mathbf{x}') + \sin(\mathbf{w}_i^T \mathbf{x})\sin(\mathbf{w}_i^T \mathbf{x}')$
  - Hence, $k(\mathbf{x}, \mathbf{x}') \approx \boldsymbol{\phi}(\mathbf{x})^T \boldsymbol{\phi}(\mathbf{x}')$ with
    $$\boldsymbol{\phi}(\mathbf{x}) = \frac{\sigma_f}{\sqrt{M}} \left[\cos(\mathbf{w}_1^T \mathbf{x}), \sin(\mathbf{w}_1^T \mathbf{x}), \ldots, \cos(\mathbf{w}_M^T \mathbf{x}), \sin(\mathbf{w}_M^T \mathbf{x})\right]^T.$$

From random features to neural networks

- **Random Fourier features:** To approximate k we compute $k(\mathbf{x}, \mathbf{x}') \approx \phi(\mathbf{x})^T \phi(\mathbf{x}')$ with
$$\phi(\mathbf{x}) = \frac{\sigma_f}{\sqrt{M}} \left[\cos(\mathbf{w}_1^T \mathbf{x}), \sin(\mathbf{w}_1^T \mathbf{x}), \ldots, \cos(\mathbf{w}_M^T \mathbf{x}), \sin(\mathbf{w}_M^T \mathbf{x})\right]^T$$
using randomly generated parameters $\mathbf{w}_1, \ldots, \mathbf{w}_M \sim \mathcal{N}(\mathbf{0}, \ell^{-2}\mathbf{I})$.

- We can write this as: $f(\mathbf{x}) = \phi(\mathbf{x})^T \mathbf{v} = \frac{\sigma_f}{\sqrt{M}} \sum_{m=1}^{M} \left(v_m^c \cos(\mathbf{w}_m^T \mathbf{x}) + v_m^s \sin(\mathbf{w}_m^T \mathbf{x})\right)$

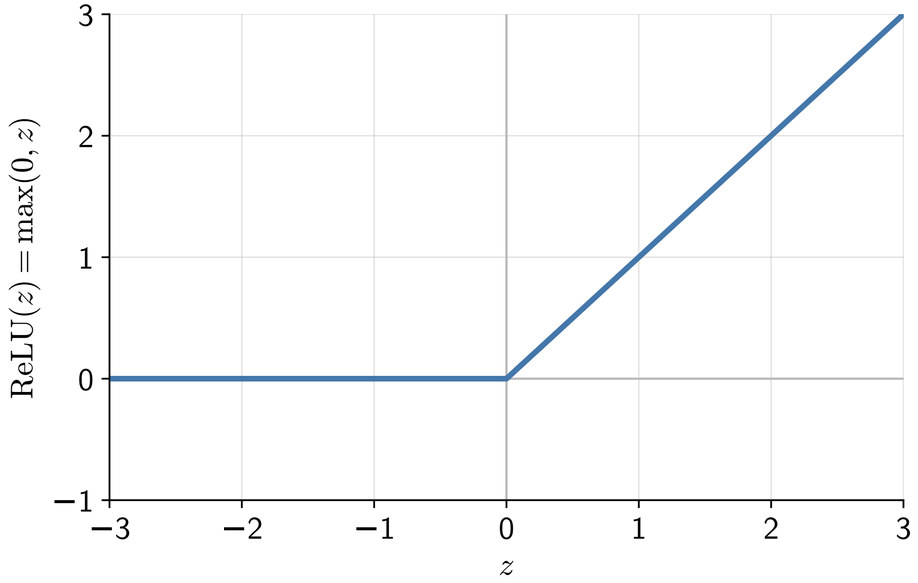
- **Observation:** f is a linear model on features $\phi(\mathbf{x})$, and the features are parameterized by $\mathbf{w}_1, \ldots, \mathbf{w}_M$, which we sample at random and keep fixed.

- **Key idea:** What if we learn the parameters $\mathbf{w}_1, \ldots, \mathbf{w}_M$ from data, together with $\mathbf{v}$?

Basics of ReLU neural networks

- A neural network is a (typically) non-linear function defined through function composition, where the non-linearity arises from so-called activation functions.

- ReLU activation function: $g(x) = \max\{x,0\}$



Basics of ReLU neural networks

- A neural network is a (typically) non-linear function defined through function composition, where the non-linearity arises from so-called activation functions.

- ReLU activation function: $g(x) = \max\{x, 0\}$

- Example neural network (toy example): $f = f_3 \circ f_2 \circ f_1$,

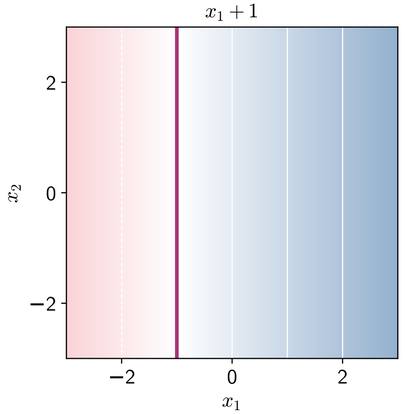
$$f_1(\mathbf{x}) = g(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$f_2(\mathbf{h}) = g(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$
$$f_3(\mathbf{h}) = \mathbf{w}_3^T\mathbf{h}$$

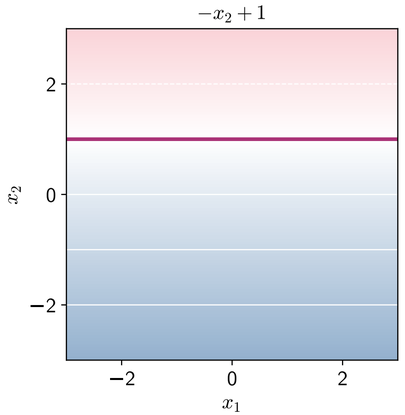
- With one hidden layer: $f(\mathbf{x}) = \sum_{m=1}^{M} v_m\, g(\mathbf{w}_m^T\mathbf{x} + b_m)$, which is the random features model, with an activation g instead of $\cos$ or $\sin$, and with learned $\mathbf{w}_m$.
  Stacking such layers gives deep networks.

Basics of ReLU neural networks

- Example neural network (toy example): $f = f_3 \circ f_2 \circ f_1,$

$$f_1(\mathbf{x}) = g(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$f_2(\mathbf{h}) = g(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$
$$f_3(\mathbf{h}) = \mathbf{w}_3^T\mathbf{h}$$

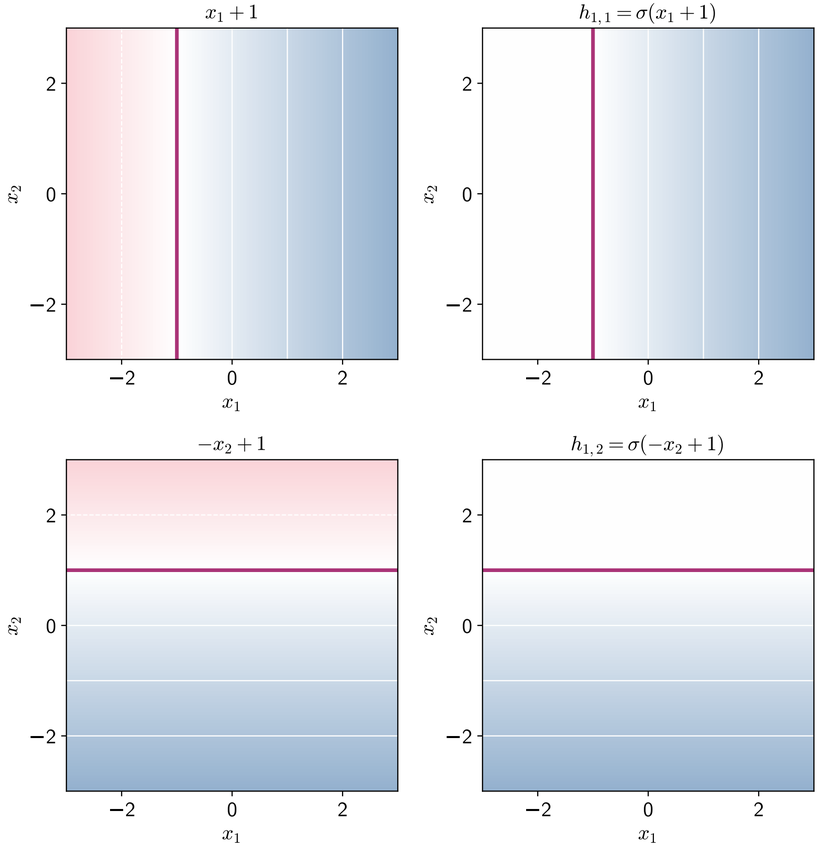




Basics of ReLU neural networks

- Example neural network (toy example): $f = f_3 \circ f_2 \circ f_1,$

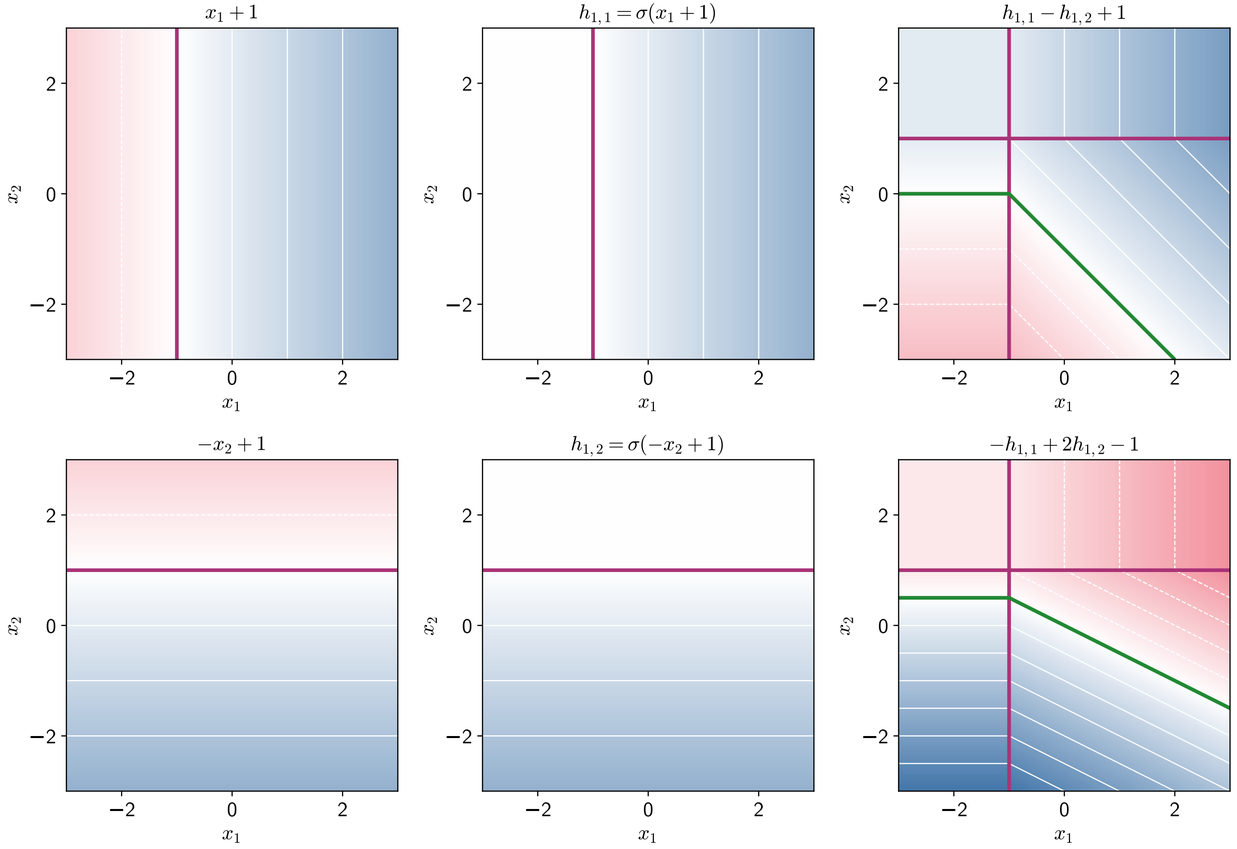
$$f_1(\mathbf{x}) = g(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$f_2(\mathbf{h}) = g(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$
$$f_3(\mathbf{h}) = \mathbf{w}_3^T\mathbf{h}$$



Basics of ReLU neural networks

- Example neural network (toy example): $f = f_3 \circ f_2 \circ f_1,$

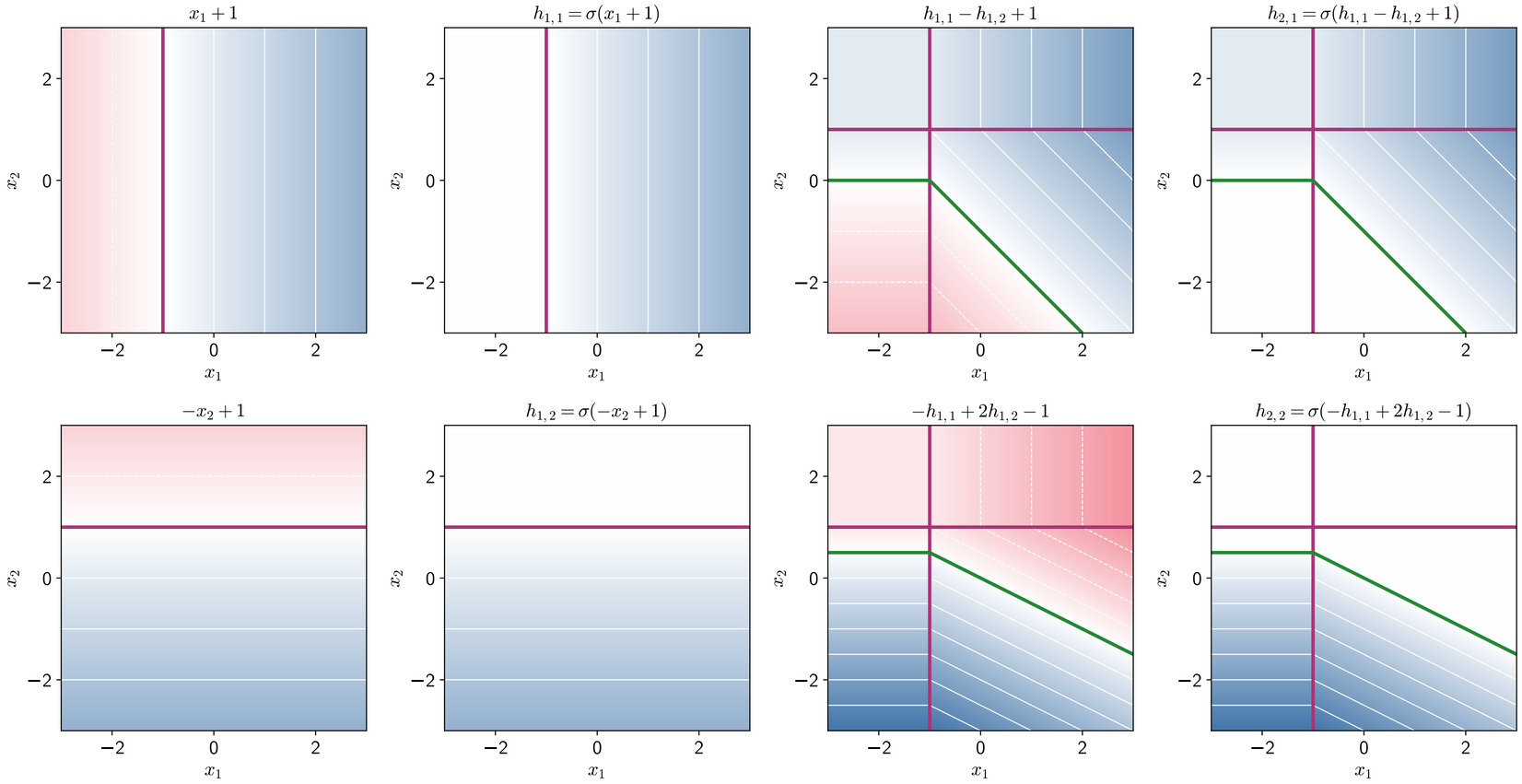
$$f_1(\mathbf{x}) = g(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$f_2(\mathbf{h}) = g(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$
$$f_3(\mathbf{h}) = \mathbf{w}_3^T\mathbf{h}$$



Basics of ReLU neural networks

- Example neural network (toy example): $f = f_3 \circ f_2 \circ f_1,$

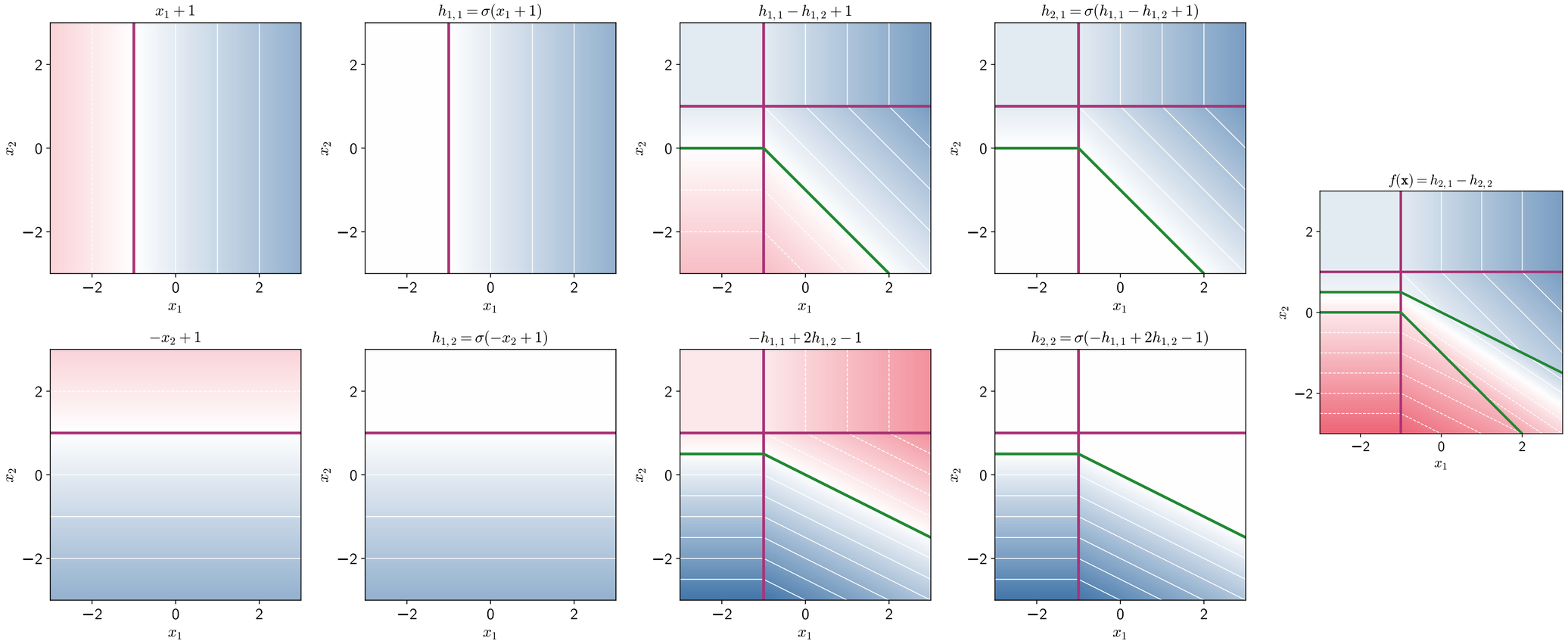
$$f_1(\mathbf{x}) = g(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$f_2(\mathbf{h}) = g(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$
$$f_3(\mathbf{h}) = \mathbf{w}_3^T\mathbf{h}$$



Basics of ReLU neural networks

- Example neural network (toy example): $f = f_3 \circ f_2 \circ f_1,$

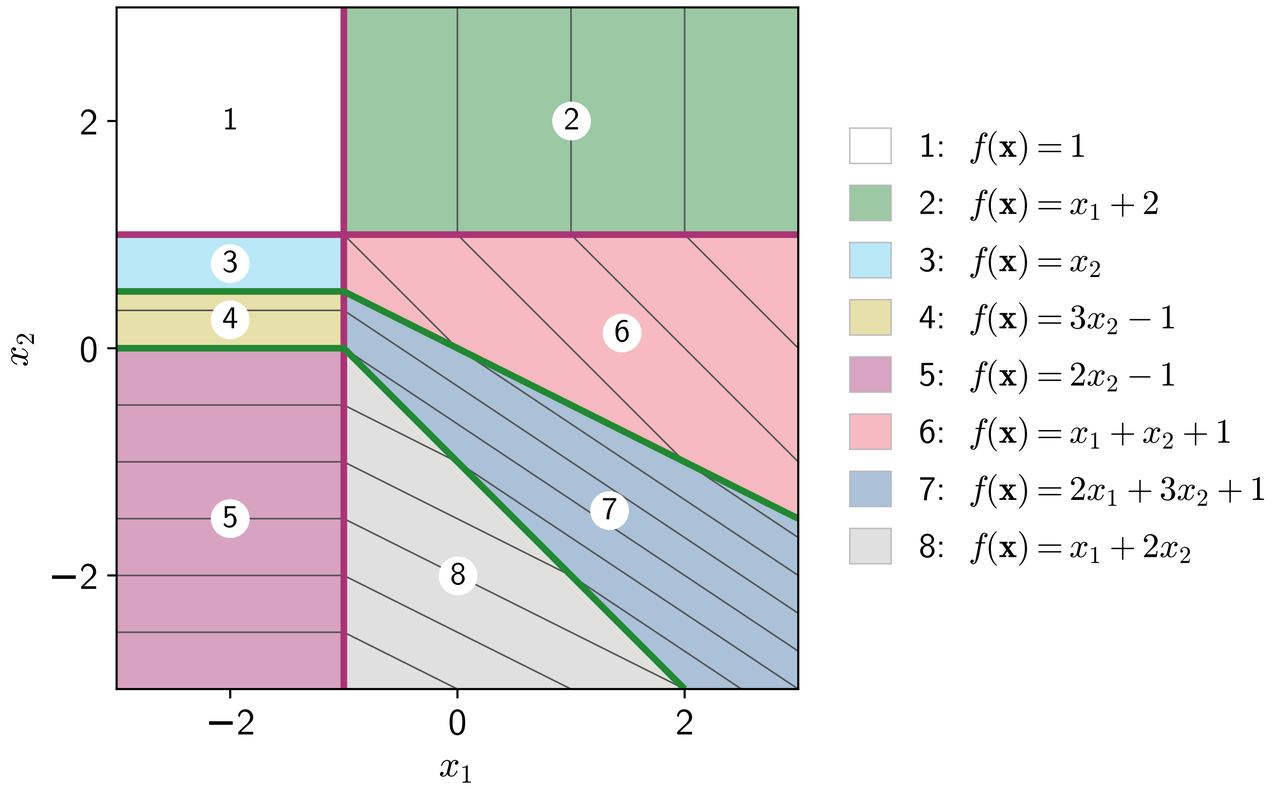
$$f_1(\mathbf{x}) = g(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$f_2(\mathbf{h}) = g(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$
$$f_3(\mathbf{h}) = \mathbf{w}_3^T\mathbf{h}$$



Basics of ReLU neural networks

- Example neural network (toy example): $f = f_3 \circ f_2 \circ f_1,$

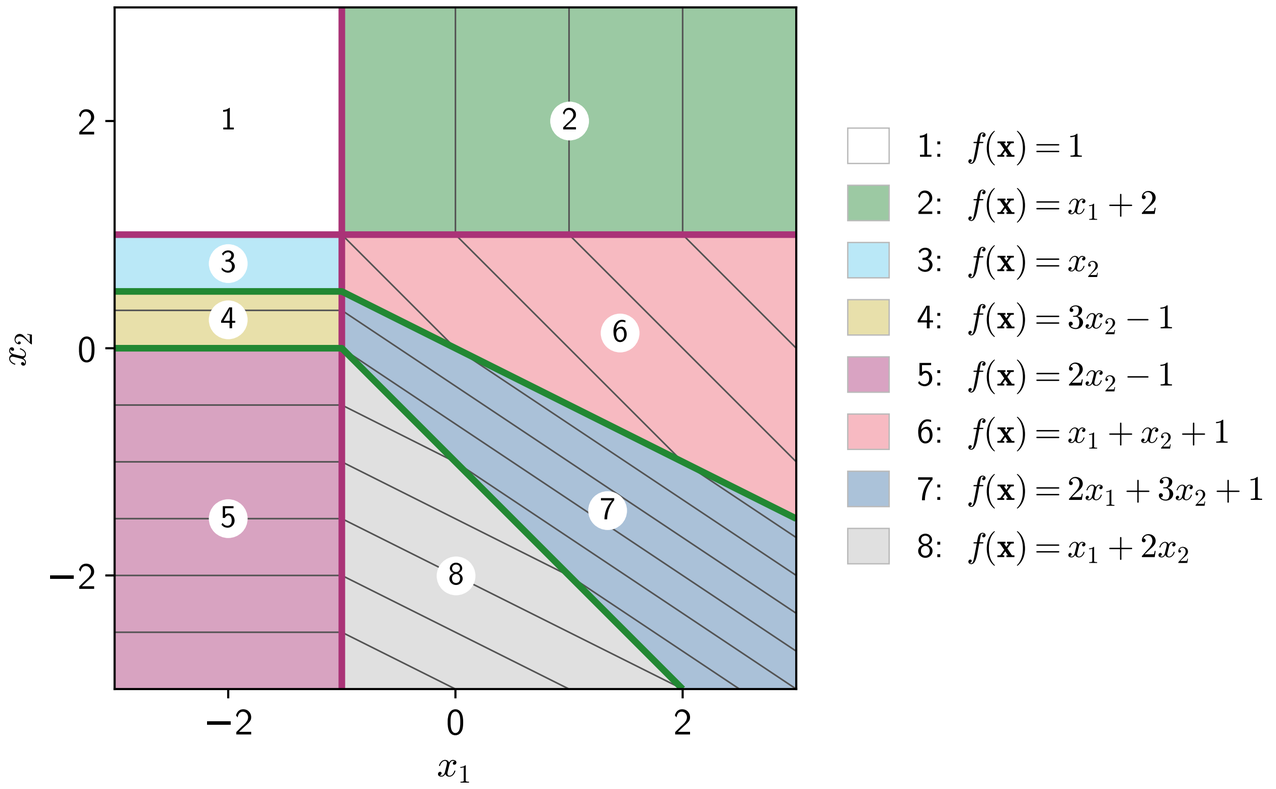
$$f_1(\mathbf{x}) = g(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$f_2(\mathbf{h}) = g(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$
$$f_3(\mathbf{h}) = \mathbf{w}_3^T\mathbf{h}$$



Basics of ReLU neural networks

- Example neural network (toy example): $f = f_3 \circ f_2 \circ f_1,$

$$f_1(\mathbf{x}) = g(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$f_2(\mathbf{h}) = g(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$
$$f_3(\mathbf{h}) = \mathbf{w}_3^T\mathbf{h}$$

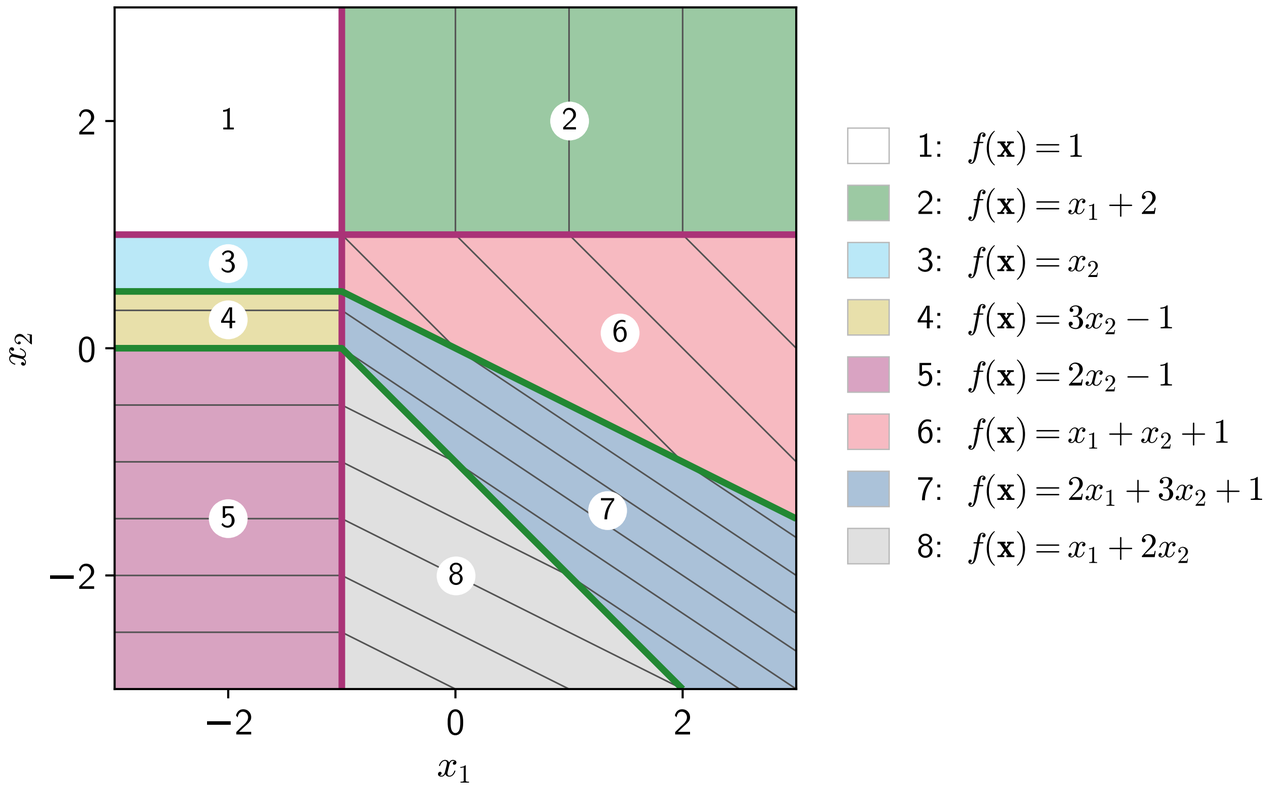


At most, how many regions do we have after f_1 with $\mathbf{W}_1 \in \mathbb{R}^{K \times D}$ and $K \leq D$?

Basics of ReLU neural networks

- Example neural network (toy example): $f = f_3 \circ f_2 \circ f_1,$

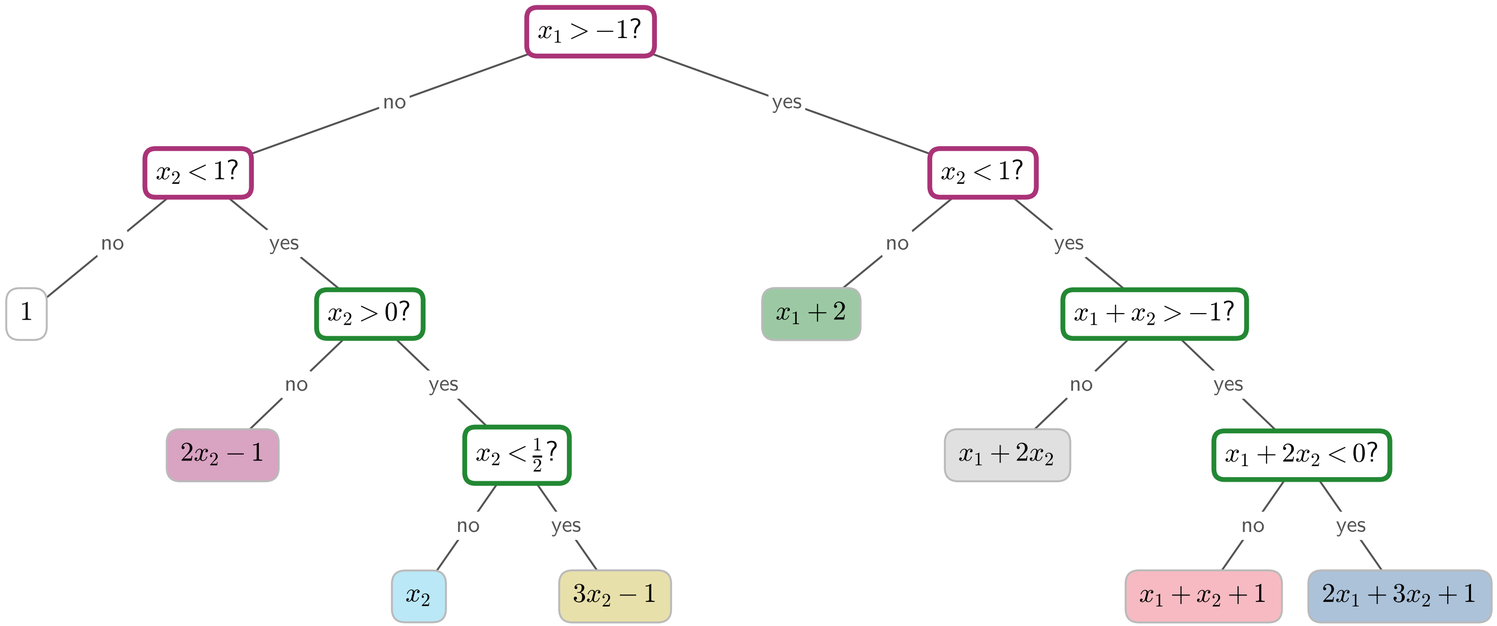
$$f_1(\mathbf{x}) = g(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$
$$f_2(\mathbf{h}) = g(\mathbf{W}_2\mathbf{h} + \mathbf{b}_2)$$
$$f_3(\mathbf{h}) = \mathbf{w}_3^T\mathbf{h}$$



At most, how many regions do we have after f_2 with $\mathbf{W}_1 \in \mathbb{R}^{K\times D}, \mathbf{W}_2 \in \mathbb{R}^{K\times K}$ and $K \leq D$?

ReLU neural networks as decision trees

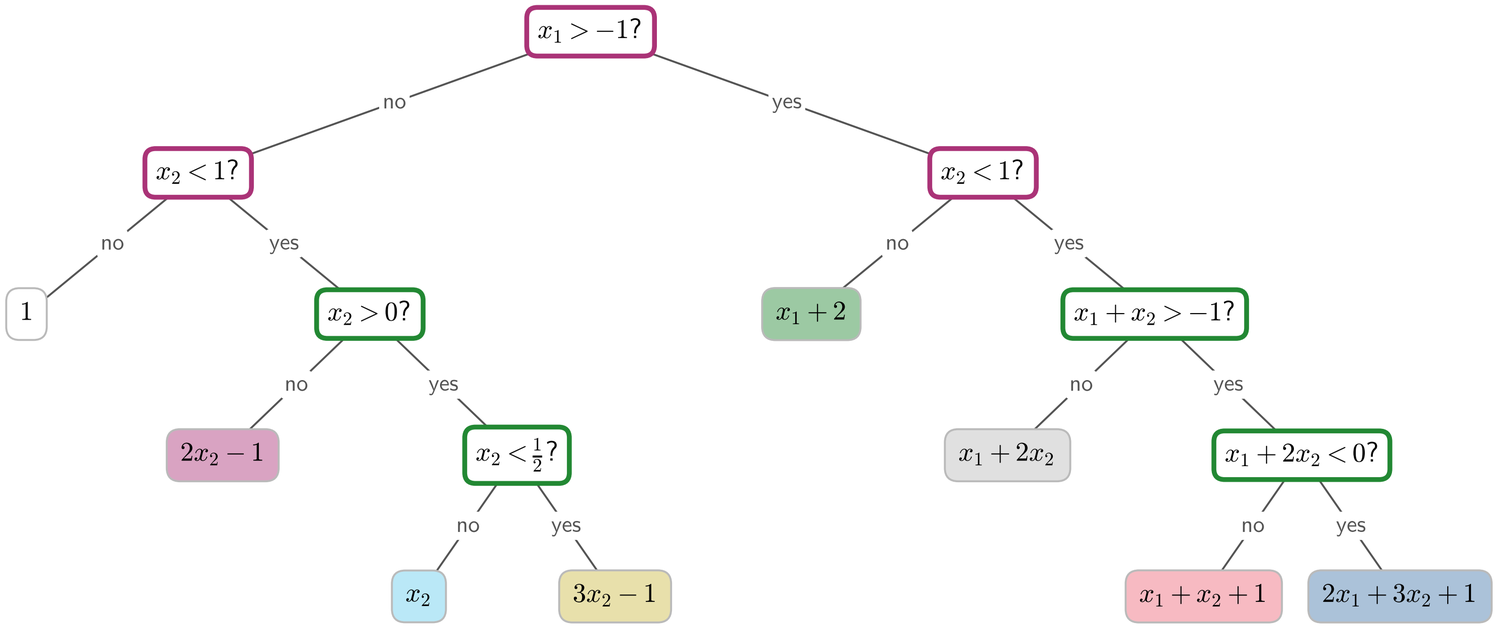
- **Observation:** ReLU neural networks can be seen as decision trees.



ReLU neural networks as decision trees

- **Observation:** ReLU neural networks can be seen as decision trees.

- **Key idea:** ReLU networks construct disjoint regions, each of which has its own locally linear function learned from data.



Implementing a ReLU neural network

Simple ReLU network

```python
import numpy as np

def g(z): return np.maximum(z, 0) # ReLU

def f1_p(W, b):
    return lambda x : g(W @ x + b)

def f2_p(w, b):
    return lambda x : w @ x + b

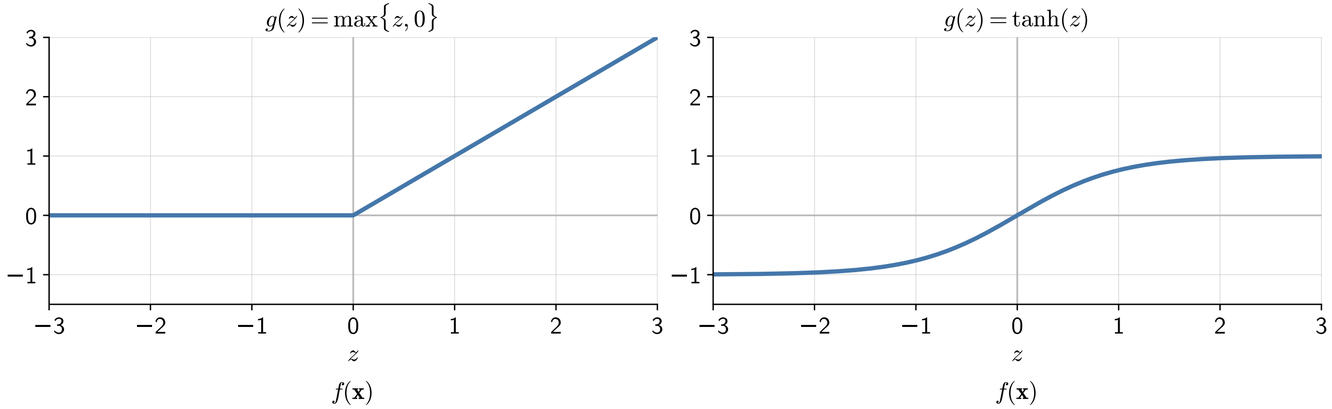
# instantiate first function with weights and biases
W1, b1 = np.array([[1.0], [1.0]]), np.array([0.0, -80.0])
f1 = f1_p(W1, b1)

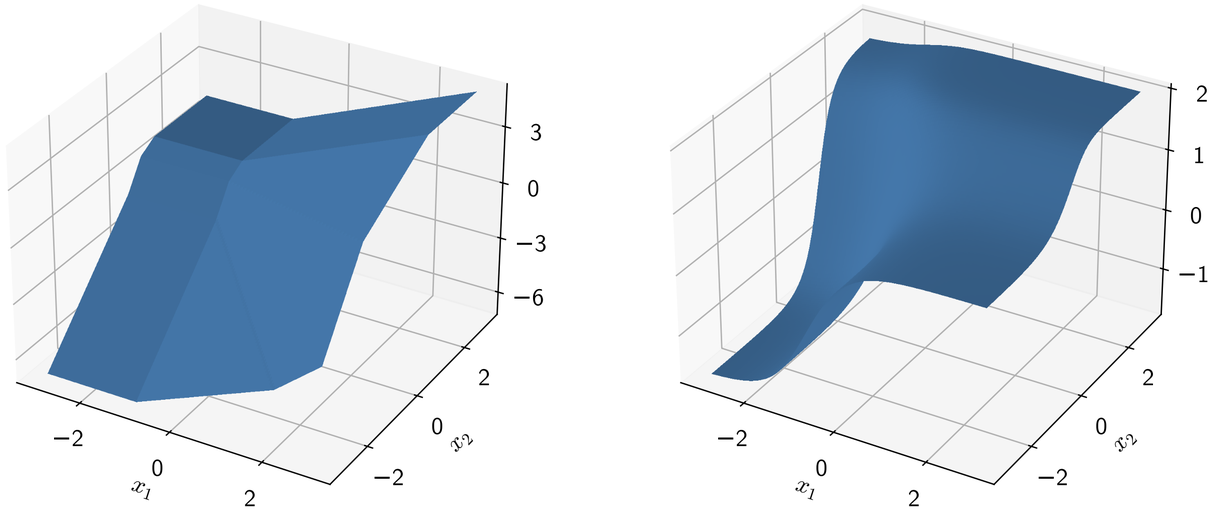
# instantiate second function with weights and biases
w2, b2 = np.array([0.09, 0.05]), 0.3
f2 = f2_p(w2, b2)

# neural network function
def f(x):
    return f2(f1(x))
```

Activation functions

- Depending on the choice of activation function $g(\cdot)$ we get different behaviour.





Activation functions

- Depending on the choice of activation function $g(\cdot)$ we get different behaviour.
- **ReLU:** $g(x) = \max\{x,0\}$**,** each unit cuts the input space with a hyperplane, and f is piecewise linear with a different linear model in each region.
- **Tanh:** $g(x) = \tanh(x)$, each unit is a soft step between -1 and 1; hence, the cuts are blurred instead of clear cuts, and f is a smooth function instead.
- **Cosine:** $g(x) = \cos(x)$, each unit adds a wave, and with random weights the network's prior over functions is stationary, i.e., its kernel depends only on $\mathbf{x} - \mathbf{x}'$ (Bochner's theorem, as for random Fourier features).

L. Meronen, M. Trapp, A. Solin. Periodic Activation Functions Induce Stationarity. NeurIPS 2021.

Neural networks as feature maps

- **Key observation:** The last layer is a linear model $f(\mathbf{x}) = \mathbf{w}_3^T \underbrace{f_2(f_1(\mathbf{x}))}_{\phi(\mathbf{x})} = \phi(\mathbf{x})^T \mathbf{w}_3$

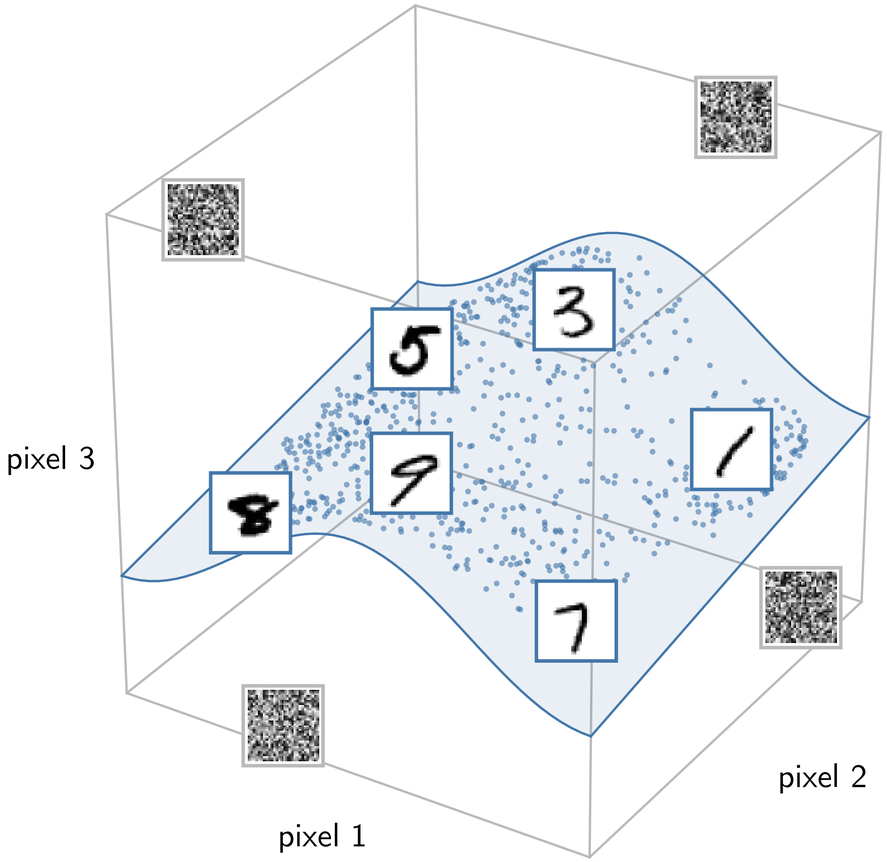
- Hence, we can see neural networks as representing a learned feature map!

- **Comparison:**
  - **Lectures 5–6:** ϕ chosen by hand or implicit in k.
  - **Random features:** ϕ sampled.
  - **Neural networks:** ϕ learned from data!

- **Question:** Why do we want to learn ϕ instead of choosing a kernel?

Manifold hypothesis

- **Manifold hypothesis:** Real-world high-dimensional data (images, audio, text) lies close to a low-dimensional manifold embedded in the input space.



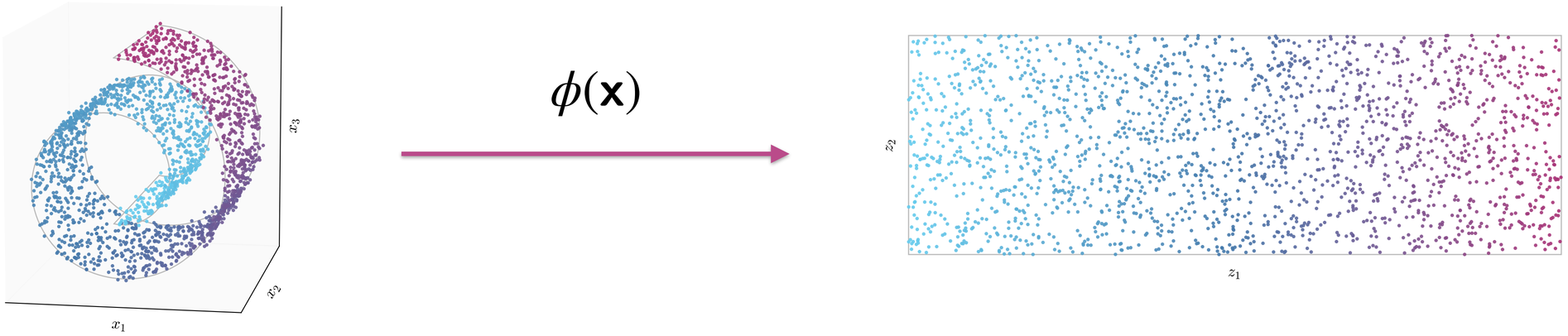
Manifold hypothesis

- **Manifold hypothesis:** Real-world high-dimensional data (images, audio, text) lies close to a low-dimensional manifold embedded in the input space.
- **Neural networks:** The hidden layers learn a map $\phi(\mathbf{x})$ from the pixel space to coordinates on the data manifold, where the manifold is (approximately) flat.
- **Predictions:** The last layer $f(\mathbf{x}) = \phi(\mathbf{x})^T \mathbf{w}_3$ is a linear model in this space, for regression or classification.
- **Key idea:** Distances and similarities between observations are measured in the learned feature space, e.g., $\|\phi(\mathbf{x}) - \phi(\mathbf{x}')\|$, instead of in the input space.

Manifold hypothesis

- **Manifold hypothesis:** Real-world high-dimensional data (images, audio, text) lies close to a low-dimensional manifold embedded in the input space.

- **Key idea:** Distances and similarities between observations are measured in the learned feature space, e.g., $\|\phi(\mathbf{x}) - \phi(\mathbf{x}')\|$, instead of in the input space.



Learning neural networks

- **Recall:** A neural network is a composition $f = f_3 \circ f_2 \circ f_1$ with parameters $\theta = \{\mathbf{W}_1, \mathbf{b}_1, \mathbf{W}_2, \mathbf{b}_2, \mathbf{w}_3\}$.

- **Goal:** Find the parameters θ that minimize the squared loss $\mathscr{L}(\theta) = \sum_{n=1}^{N} \big(y_n - f(\mathbf{x}_n; \theta)\big)^2$.

- **Recall (lectures 5–6):** In linear and ridge regression, f is linear in the parameters, so $\mathscr{L}$ is a convex quadratic, and we can compute the optimal solution analytically.

- **Problem:** $f(\mathbf{x}; \theta)$ is not linear in the parameters θ due to function composition and the activation function g, hence, $\mathscr{L}(\theta)$ is not convex, and no closed-form solution exists.

Learning neural networks

- **Key idea:** Instead of solving for the optimal θ analytically, we use gradient descent and compute the gradient $\nabla_\theta \mathscr{L}(\theta)$ of the loss with respect to the parameters using the **chain rule.**
- **Chain rule:** For $f(\mathbf{x}) = a(b(\mathbf{x}))$, we have $\dfrac{\partial f}{\partial \mathbf{x}} = \dfrac{\partial a}{\partial b}\dfrac{\partial b}{\partial \mathbf{x}}$.
- **Toy network:** With $\mathbf{h}_1 = f_1(\mathbf{x}; \mathbf{W}_1, \mathbf{b}_1)$, $\mathbf{h}_2 = f_2(\mathbf{h}_1; \mathbf{W}_2, \mathbf{b}_2)$, and $f = f_3(\mathbf{h}_2; \mathbf{w}_3)$:

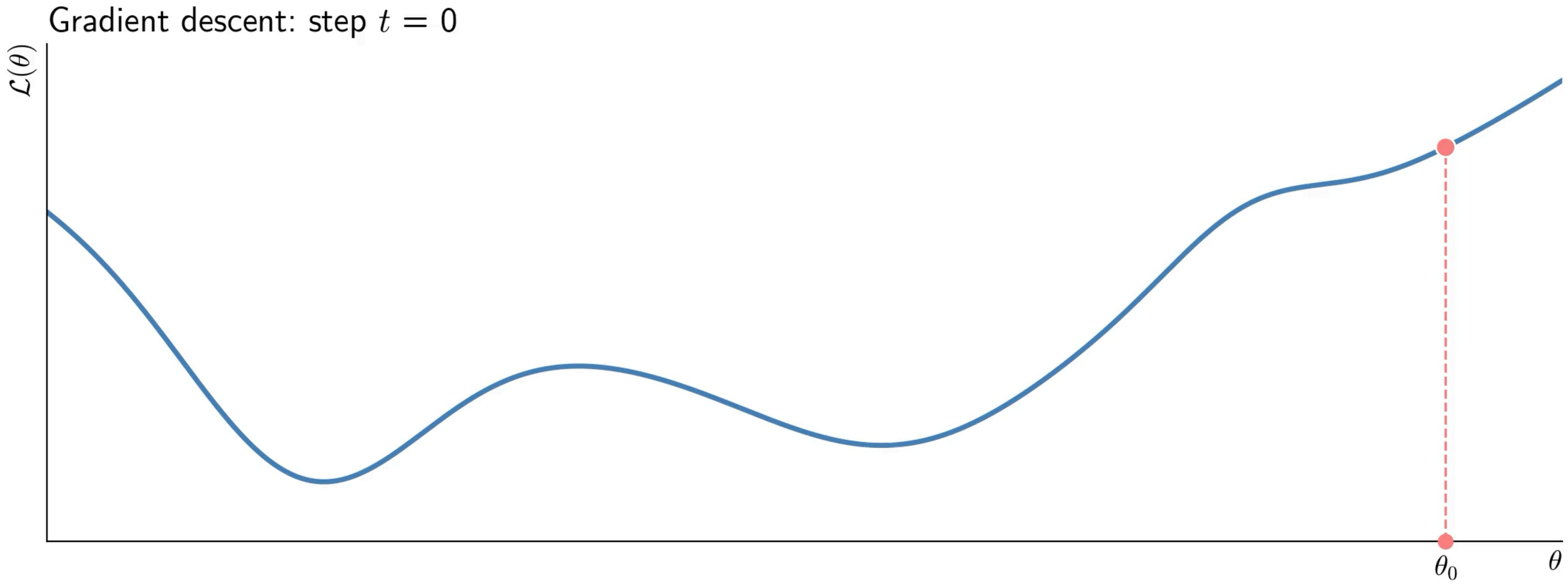
$$\frac{\partial \mathscr{L}}{\partial \mathbf{w}_3} = \frac{\partial \mathscr{L}}{\partial f}\frac{\partial f}{\partial \mathbf{w}_3}$$

$$\frac{\partial \mathscr{L}}{\partial \mathbf{W}_2} = \frac{\partial \mathscr{L}}{\partial f}\frac{\partial f}{\partial \mathbf{h}_2}\frac{\partial \mathbf{h}_2}{\partial \mathbf{W}_2}$$

$$\frac{\partial \mathscr{L}}{\partial \mathbf{W}_1} = \frac{\partial \mathscr{L}}{\partial f}\frac{\partial f}{\partial \mathbf{h}_2}\frac{\partial \mathbf{h}_2}{\partial \mathbf{h}_1}\frac{\partial \mathbf{h}_1}{\partial \mathbf{W}_1}$$

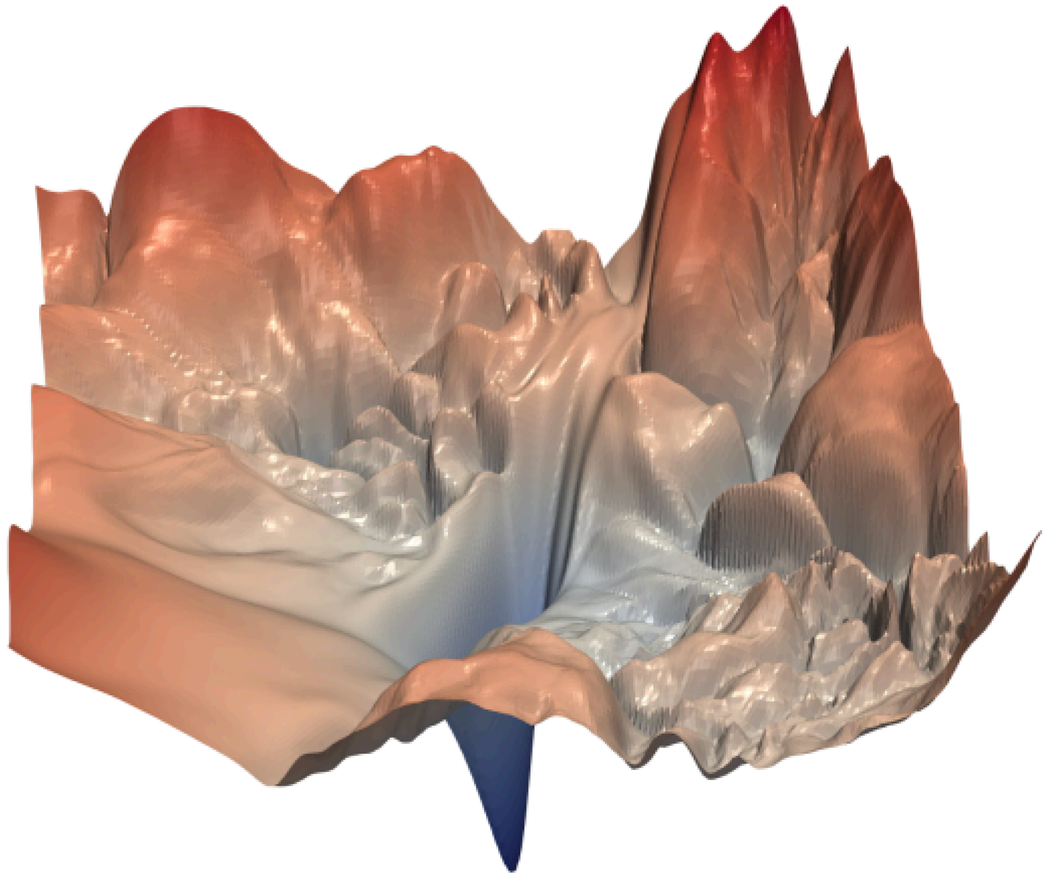
Gradient descent

- **Gradient descent:** Starting from an initial θ, we repeatedly take a small step in the direction of the **negative gradient** of the loss $\theta \leftarrow \theta - \eta \nabla_\theta \mathscr{L}(\theta)$.



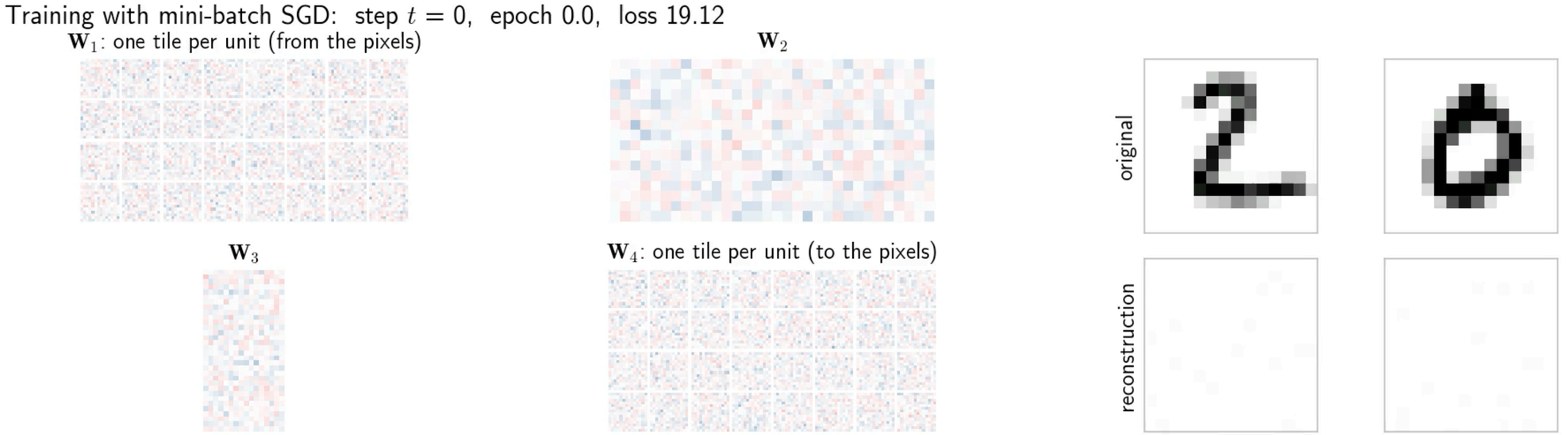
Gradient descent - neural network example

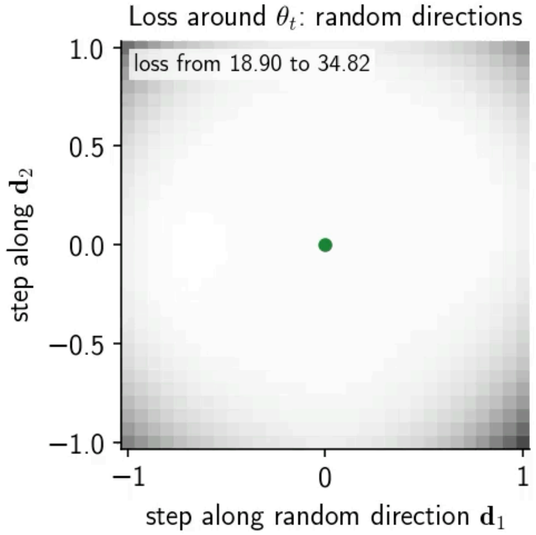
- Illustration of the loss landscape of neural networks.

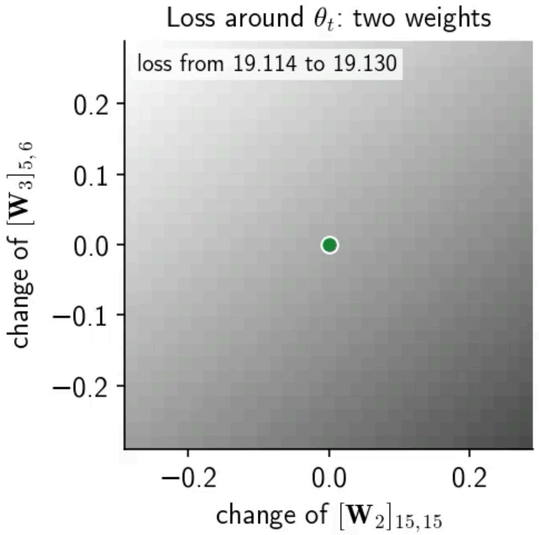


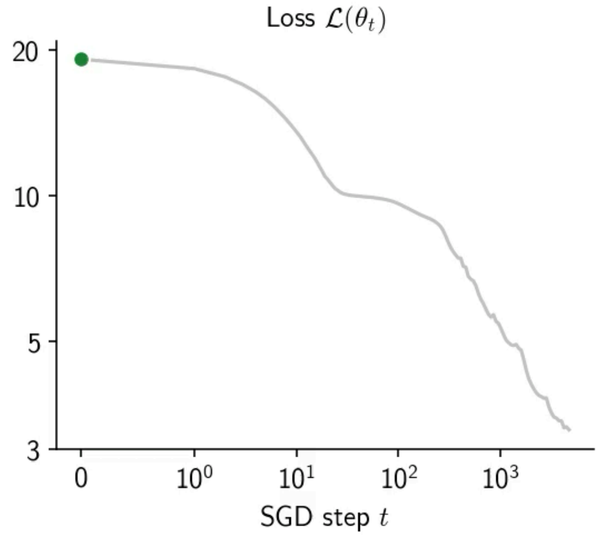
H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein. Visualizing the Loss Landscape of Neural Nets. NeurIPS 2018.

Gradient descent - neural network example

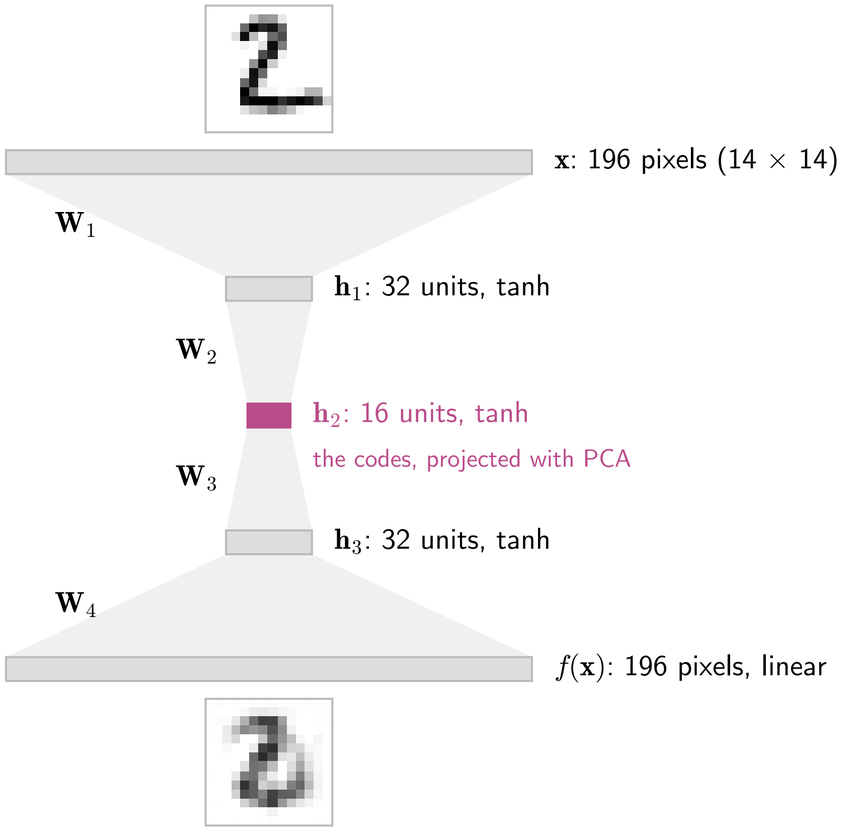


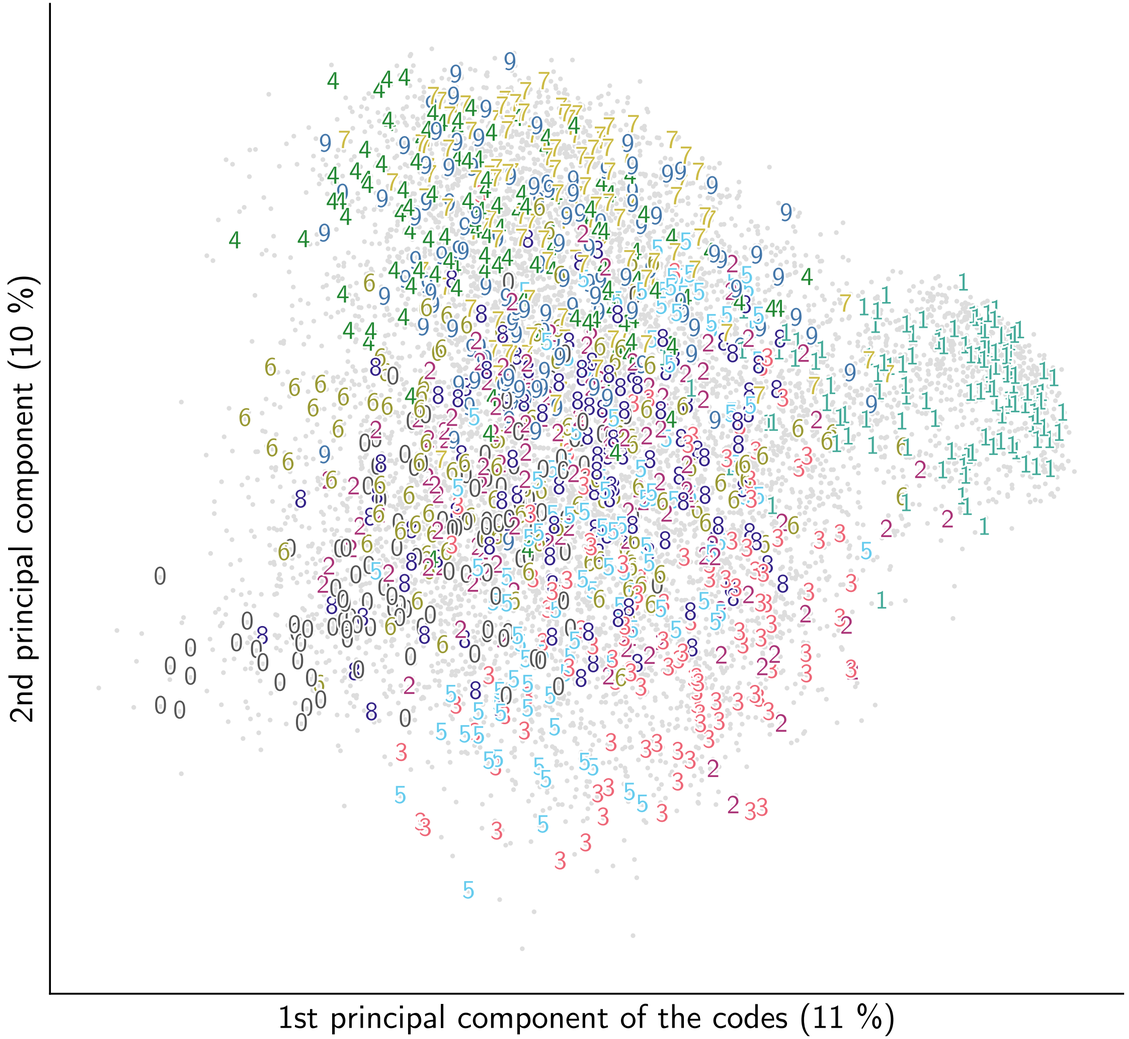






What has the network learned?





The end

Next lecture: Using learned feature maps in Gaussian processes and other models.

